

Logistic Classifiers

Binary logistic regression

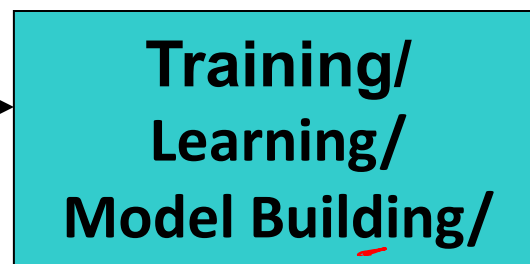
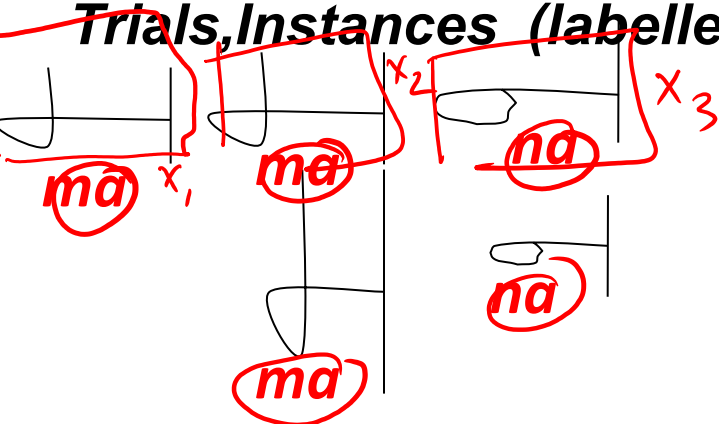
Multiclass logistic regression

Classification

 *ML component*

TRAINING

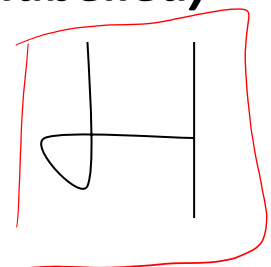
*Inputs, Examples,
Trials, Instances (labelled)*



Classifier/
Function/
Predictor/
Model

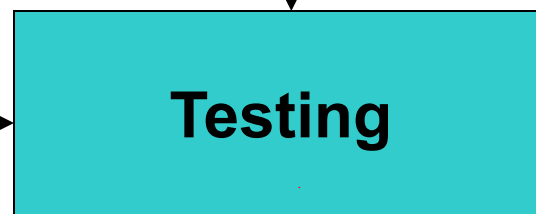
TESTING

*Test example
(unlabelled)*



Trained Model

read model



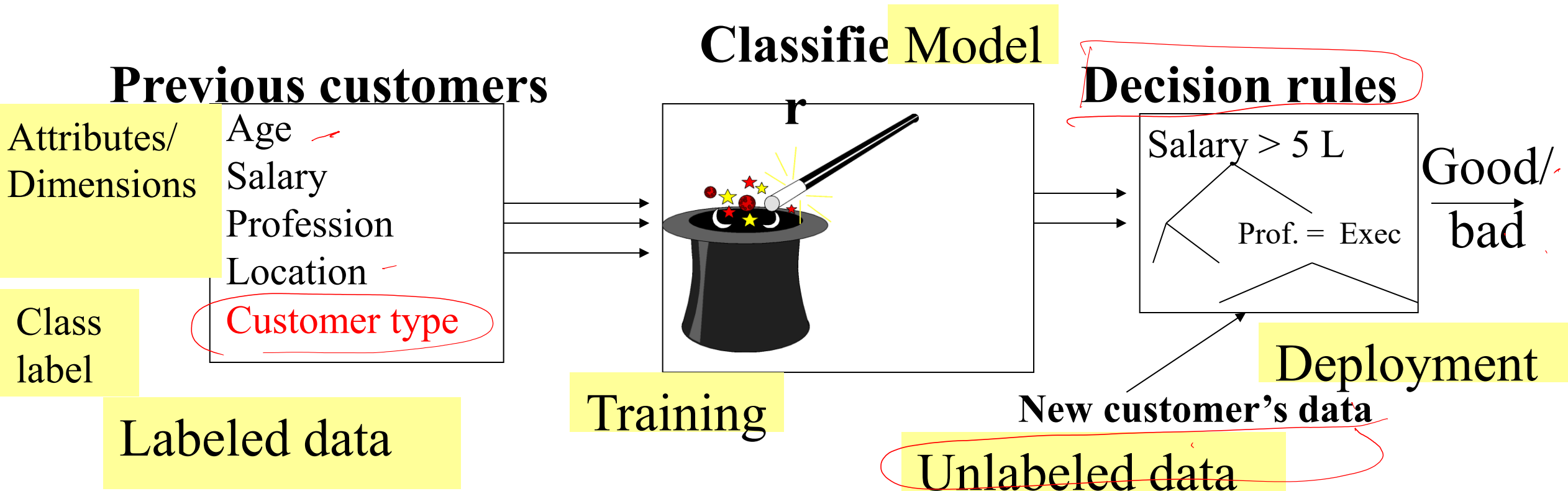
Predicted
label *ma*

Performance and
reward/loss (0/1 loss,
recall, precision, F1, ROC,
AUC)

True label

Classification example

- Given old data about customers and payments, predict new applicant's loan eligibility.

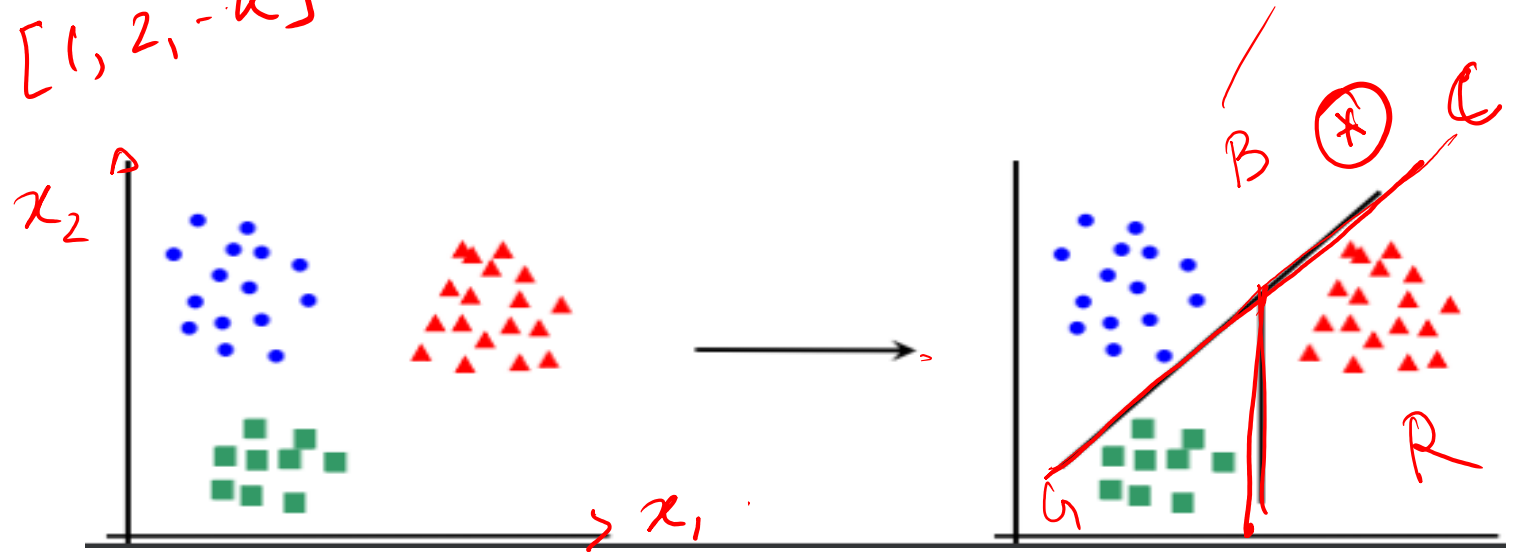


Classification: setup

- Input: Like in regression, obtained after feature engineering $x \in R^d$
- Output: one of K possible class labels $y \in [1, 2, \dots, K]$ and/or a distribution over class labels.
- Training data:

$$\begin{matrix} X & Y \\ \begin{bmatrix} x_1^1 & x_2^1 & \dots & x_d^1 \\ \vdots & \vdots & & \vdots \\ x_1^N & x_2^N & \dots & x_d^N \end{bmatrix} & \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix} \end{matrix}$$

$y_i \in [1, 2, \dots, K]$

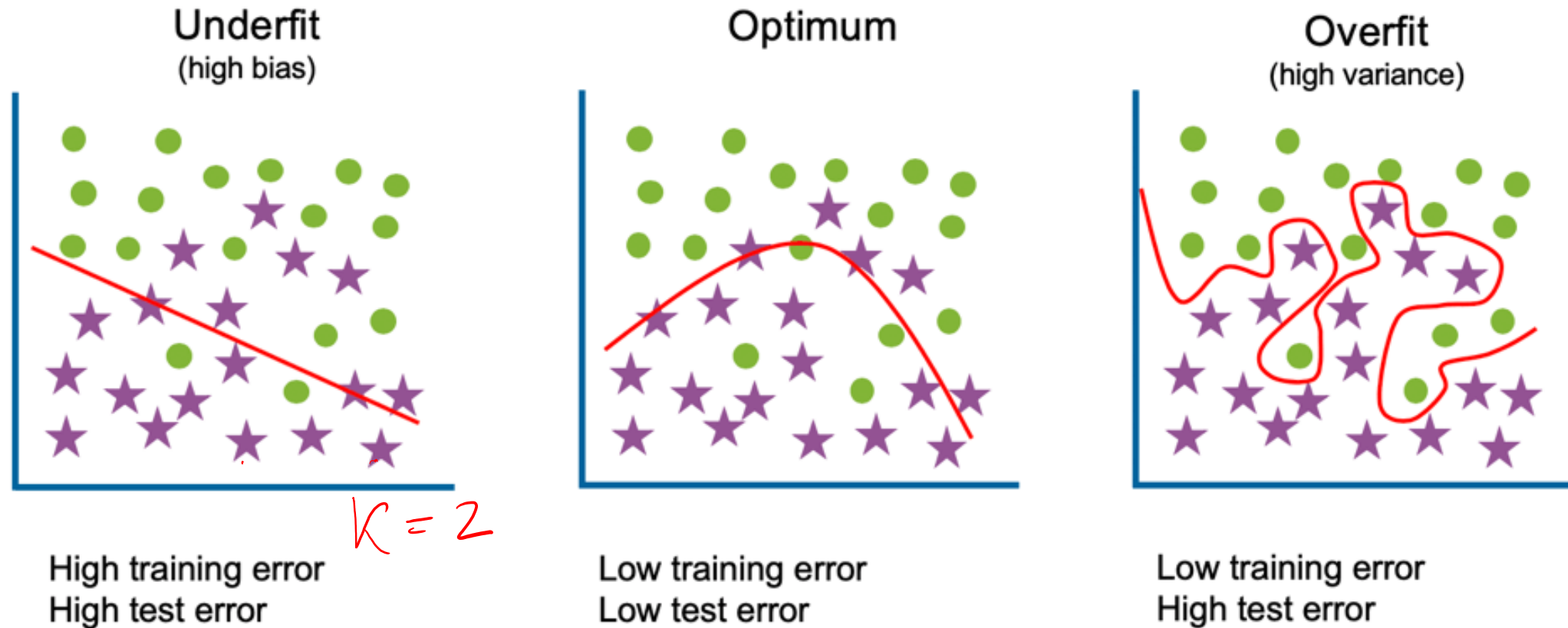


An example training dataset $d=2, K=3$

Classifier: $C(x) \rightarrow \hat{y}$

$$C(x) \rightarrow P(y|x) = \begin{bmatrix} 0.1 \\ 0.2 \\ \vdots \\ 0.01 \end{bmatrix} \} K \text{ such probabilities}$$

Over-fitting in classifiers



Types of classifiers

- Discriminative
 - Logistic regression /
 - Decision trees |
 - Neural Network |
- Generative classifiers
 - Naïve Bayes, LDA, etc. ||
- Kernel-based
 - Nearest neighbor classifier,
 - Support Vector Machines

Logistic ^{regression} classifiers

- • What is the hypothesis class?
- What is the training objective?
- How to prevent over-fitting?
- How to efficiently train the classifier?

Linear Functions as Hypothesis Class:

1) First we choose a feature map or basis transformation of x

$\phi(x)$ $\rightarrow$ q -dimensional real vector. x $\equiv$ $\begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix}$

Examples:

1) $\phi(x) = x$ $\leftarrow$ Linear map

$$\underline{\phi(x)} = \begin{bmatrix} x_1 \\ x_1^2 \\ x_1^3 \\ \vdots \\ x_1^p \end{bmatrix}$$

polynomial features
 p

$$\underline{d = 1}$$

other examples: Gaussian kernels,
Example of hypothesis class: polynomials of degree p where p is any value from $\{1, \dots, 100\}$

Recap

$$\mathcal{F} = \{ \underline{w}^T \phi(x) + \underline{w}_0 : \forall w \in \mathbb{R}^q ; w_0 \in \mathbb{R} \}$$

Example

$$q = 2$$

$$w =$$

$$\begin{bmatrix} w_1 \\ \vdots \\ w_q \end{bmatrix}$$

$$= w_1 \phi_1(x) + w_2 \phi_2(x) + w_0$$

$$\mathcal{F} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}^T \begin{bmatrix} \phi_1(x) \\ \phi_2(x) \end{bmatrix} + w_0$$

$$f(x, \theta) \equiv \sum_{j=1}^q w_j \phi_j(x) + w_0$$

$$\begin{aligned} \theta &\equiv \{w_1, \dots, w_q, w_0\} \\ &= \{w, w_0\} \end{aligned}$$

2) Choose error function as square error:

$$l(\hat{y}, y) = (\hat{y} - y)^2$$

Logistic classifiers

- Define a feature map exactly like for linear regression $x \in \mathbb{R}^d$
 $\phi(x) \in \mathbb{R}^2$

For simplicity of notation: henceforth we drop $\phi(x)$ and overload x to denote $\phi(x)$

- For each class k, define a linear function, also called logits or scoring function

$$\underline{f_k(x)} = \underline{w_{k1}x_1 + \dots + w_{kd}x_d + w_{k0}} = \underline{W_k^T x + w_{k0}}$$

Assume that x includes one dimension that is a constant "1" for all instances.
So we drop w_{k0} for notational convenience often.
 $f_k(x) = W_k^T x$

- Assign predicted label $\hat{y}$ as the class for which score is maximized.

$$c(x) = \underset{1 \leq c \leq K}{\operatorname{argmax}} \underbrace{f_c(x)}_{\triangleq a_c \text{ in the textbook}} = \hat{y}$$

- Define a probability distribution over the possible classes as a multinomial distribution

$$p(y|x) \sim \text{Multinomial distribution} \quad \begin{cases} \mu_1, \mu_2, \dots, \mu_K \\ \mu_c \geq 0 \end{cases} \quad \sum_{c=1}^K \mu_c = 1$$

$$\mu_c \propto e^{f_c(x)} = e^{a_c}$$

$$\sum_c \mu_c = 1 \Rightarrow \mu_c = \frac{e^{a_c}}{\sum_{c'=1}^K e^{a_{c'}}}$$

$$\frac{e^{w_c^T x}}{\sum_{c'=1}^K e^{w_{c'}^T x}}$$

The above is called softmax

$$\text{Softmax} \left(\begin{matrix} a_1 \\ \vdots \\ a_k \end{matrix} \right) = \begin{bmatrix} \frac{e^{a_1}}{Z} \\ \vdots \\ \frac{e^{a_k}}{Z} \end{bmatrix} \quad \text{where } Z = \sum_c e^{a_c}$$

logits.

$\mathbb{R}$ probabilities

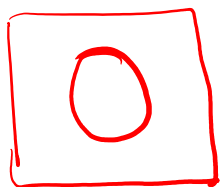
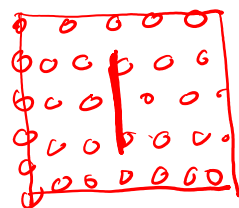
$$\log u_c = a_c - \log Z$$

Demo

- <https://colab.research.google.com/drive/1MtvbOlHgmkV05GJXfefisim6TiClEXts?usp=sharing>

Example: Digit recognition: classes {0,1,4}

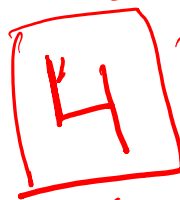
Features:



x^2



x^3



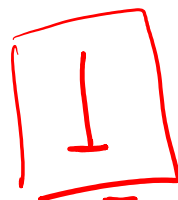
x^4

$x_1 \equiv \# \text{ of vertical lines}$

$x_2 \equiv \# \text{ of circles}$

$x_3 \equiv \# \text{ of horizontal lines}$

$x_4 \equiv \# \text{ of slanted lines}$



x^5

$x^1: [1, 0, 0, 0]$ $x^2: [0, 1, 0, 0]$

$x^3: [1, 0, 1, 1]$

$x^4: [2, 0, 1, 0]$

$x^5: [1, 0, 1, 0]$

$w_{11}, w_{12}, w_{13}, w_{14} = [10, -1, -1, -1], w_{10} = 0$

$w_{21}, w_{22}, w_{23}, w_{24} = [-1, 10, -1, -1], w_{20} = 0$

$w_{31}, w_{32}, w_{33}, w_{34} = [5, 0, 10, 5], w_{30} = 0$

$$f_1(x^4) \equiv 18$$

$$f_2(x^4) \equiv -1$$

$$f_3(x^4) \equiv 20$$

$$\operatorname{argmax}_{k \in \{1, 2, 3\}} (f_k(x)) = \hat{y} = 3$$

$$y = 3$$

$$P(y = 3 | x^4) = \frac{e^{20}}{e^{18} + e^{20} + e^{-1}} \approx 22$$

Training

- Given labelled examples

$$\rightarrow D = \{(x^1, y_1), \dots, (x^N, y_N)\}$$

- Find parameters so as to maximize the probability of observing the correct class

$$\max_{\underbrace{w^1, \dots, w^K}_W} \sum_{i=1}^N \log P(y_i | x^i, w) = \frac{e^{w_{y_i}^T x^i}}{\sum_{c=1}^K e^{w_c^T x^i}}$$

Express y_i as a one-hot vector representation

$$y_i = 3 \quad \Rightarrow \quad y_i = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

a class labeled c will be a K dimensional binary vector with 1 at position c and 0 elsewhere

Rewriting the objective using 1-hot representation of class label.

$$\max_{\underbrace{w_1, \dots, w_n}_W} \sum_{i=1}^N \sum_{c=1}^K y_{ic} \log u_{ic} \quad \text{where} \quad u_{ic} = \frac{e^{w_c x^i}}{\sum_{c'=1}^K e^{w_{c'} x^i}}$$

$$\equiv \min_W \left[\sum_{i=1}^N \sum_{c=1}^K y_{ic} \log u_{ic} \right]$$

NLL = Negative log likelihood

cross-entropy

$$H(p, q) = - \sum_{j=1}^K p_j \log q_j \quad \text{where} \quad \sum_{i=1}^K p_i = 1 \quad \sum_{i=1}^K q_i = 1 \quad p_i, q_i \geq 0$$

discrete distribution over $[1 \dots K]$

$$H(q, q) = \text{entropy of } q$$

given a 'p' for what q HW is $H(p, q)$ minimized?

$$\min_{a_1, \dots, a_K} - \sum_{i=1}^K p_i \log q_i \quad \text{s.t. } q_i \geq 0 \quad \sum_{i=1}^K q_i = 1$$

optimum q is "p"

Training objective:

$$\min_W \sum_{i=1}^N H(\underline{\vec{y}}_i, \underline{\vec{u}}_i)$$

$$\underline{\vec{u}}_i = \begin{bmatrix} \frac{e^{w_1 x^i}}{z} \\ \vdots \\ \frac{e^{w_k x^i}}{z} \end{bmatrix}$$

$\underline{\vec{u}}_i$

$k=5$

$$\begin{bmatrix} 0.1 \\ 0.9 \\ 0 \\ 0 \\ 0 \end{bmatrix} \rightarrow \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Training algorithm of MC LR

Training objective:

$$NLL(W) = -\sum_{i=1}^N \sum_{c=1}^K y_{ic} \log \mu_{ic}$$

Convex in W
 W are unrestricted

$$\mu_{ic} = \frac{e^{a_{ic}}}{\sum_{c'} e^{a_{ic'}}}$$

$$a_{ic} = W_c^T X_i$$

$$\nabla_{W_j} NLL(W) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^K \nabla_{W_j} [y_{ic} (a_{ic} - \log \sum_{c'} e^{a_{ic'}})]$$

$$= -\frac{1}{N} \sum_{i=1}^N \left(y_{ij} X_i - \nabla_{W_j} \log \sum_{c'} e^{a_{ic'}} \left[\sum_{c=1}^K y_{ic} \right] \right) = 1$$

independent of c

$$= -\frac{1}{N} \sum_{i=1}^N y_{ij} X_i - \frac{1}{\sum_{c'} e^{a_{ic'}}} \nabla_{W_j} e^{a_{ij}} = \frac{a_{ij} X_i}{\sum_{c'} e^{a_{ic'}}} = \mu_{ij} X_i$$

$$= -\frac{1}{N} \sum_{i=1}^N (y_{ij} - \mu_{ij}) X_i$$

$$\nabla_{W_j} \text{NLL}(W) = \frac{1}{N} \sum_{i=1}^N \underbrace{(u_{ij} - y_{ij})}_{\text{error on the } x^i \text{ example's } j^{\text{th}} \text{ class prediction}} x^i$$

Gradient descent algorithm:

$$W = 0$$

for $t=1$ to stopping criteria {
 $W_j^{t+1} = W_j^t - \eta_t (u_{ij}^t - y_{ij}) x^i : j=1 \dots K.$
}

suppose $y_{ij} = 1$ and $u_{ij}^t = 0.1$

Add to W_j a positive weighted contribution

suppose $t=1$ $W_j^1 = \eta_1 \times 0.9 \times x^i$
 so, for next iteration: $\underline{W_j^1 \cdot x^i} > \underline{W_j^0 \cdot x^i}$

Contrast with updates of linear regression models
where $y_i \in \mathbb{R}$

$$w^{t+1} \leftarrow w^t - \eta_t (y_i - \hat{y}_i^t) x^i$$

$$\hat{y}_i^t = (w^t)^T x^i$$

Consider the special case where $k=2$
Binary logistic regression classifiers.

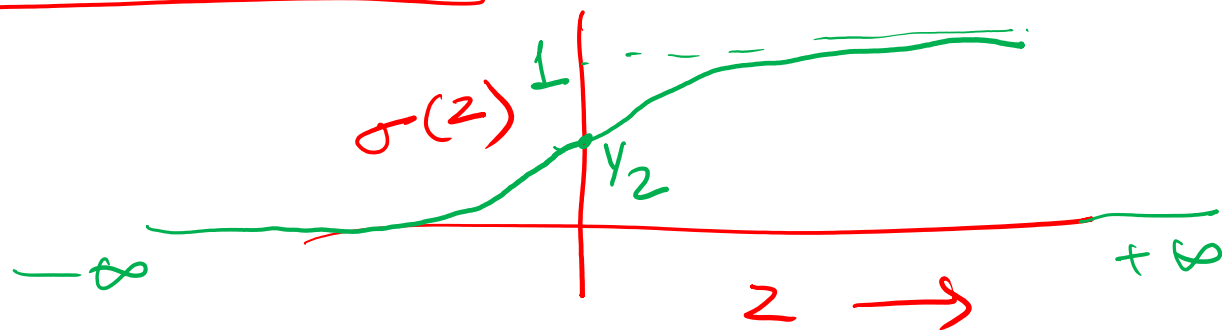
$$P(\vec{y}_i = \begin{bmatrix} 0 \\ 1 \end{bmatrix} | x^i) = \frac{e^{w_2^T x^i}}{e^{w_1^T x^i} + e^{w_2^T x^i}} = \frac{e^{(w_2 - w_1)^T x^i}}{1 + e^{(w_2 - w_1)^T x^i}}$$
$$= \frac{e^{w^T x^i}}{1 + e^{w^T x^i}} \quad \text{where } w = w_2 - w_1$$

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \sigma(w^T x^i)$$

sigmoid function.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\begin{aligned} z \rightarrow -\infty, \sigma(z) &\rightarrow 0 \\ z \rightarrow \infty, \sigma(z) &\rightarrow 1 \end{aligned}$$



change of notation for binary classification.

$$\vec{y} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \Rightarrow y = 1 \leftarrow \text{true class}$$

$$\vec{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \Rightarrow y = 0 \leftarrow \text{false class}$$

$$y \in \{0, 1\}$$

$$P(y=1|x) = \frac{1}{1 + e^{-w^T x}} \quad ; \quad p(y=0|x) = 1 - \frac{1}{1 + e^{-w^T x}} = \frac{e^{-w^T x}}{1 + e^{-w^T x}} = (1 - u)$$

$\underbrace{\qquad\qquad\qquad}_{\mu}$

Training objective:

$$NLL(w) = \frac{1}{N} \sum_{i=1}^N \left[- (y_i \log \mu_i + (1 - y_i) \log (1 - \mu_i)) \right]$$

Binary cross entropy between

$$y_i \text{ and } \mu_i$$

$$y_i \in \{0, 1\}$$

Compare with
multiclass
objective.

$$-\sum_i \sum_c y_{ic} \log \mu_{ic}$$

$y_i \sim \text{Bernoulli Distribution}(\mu_i)$

$$P(y_i | \mu_i) = \mu_i^{y_i} (1 - \mu_i)^{1 - y_i}$$

Gradient of Binary cross entropy loss:

$$\nabla_w \text{NLL}(w) = \frac{1}{N} \sum_{i=1}^N (\mu_i - y_i) x^i = \frac{1}{N} \left(\mathbf{1}_N^T \text{diag}(\vec{\mu} - \vec{y}) X_{N \times d} \right)^T$$

$$\text{diag} \left(\begin{bmatrix} z_1 \\ \vdots \\ z_N \end{bmatrix} \right) = \begin{bmatrix} z_1 & & 0 \\ & z_2 & \\ 0 & & \ddots \\ & & & z_N \end{bmatrix}_{N \times N}$$

N-rows

$$\begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix} \begin{bmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_N \end{bmatrix} \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix}$$

Hessian of the objection:

$$H(w) = \nabla_w \nabla_w^T \text{NLL}(w) = \nabla_w \left[\frac{1}{N} \sum_{i=1}^N (u_i - y_i) x_i^T \right]$$

$$u_i = \frac{1}{1 + e^{-w^T x_i}}$$

$$= \frac{1}{N} \sum_{i=1}^N (\nabla_w u_i) x_i^T$$

$$= \frac{1}{N} \sum_{i=1}^N u_i (1 - u_i) x_i x_i^T$$

$$= \frac{1}{N} X^T S X \quad \text{where } S = \text{diag} \begin{pmatrix} u_1(1-u_1) \\ \vdots \\ u_N(1-u_N) \end{pmatrix}$$

Show that the positive semi-definite -

$$\begin{aligned} \text{i.e. } \forall z \quad z^T H(w) z &\geq 0 \Rightarrow z^T X^T S X z \geq 0 \quad \forall S_{ii} \geq 0 \\ &= \left(z^T X^T S^{1/2} \right) \left(S^{1/2} X z \right) = \left\| \underbrace{z^T X^T S^{1/2}}_v \right\|_2^2 \geq 0 \Rightarrow \sqrt{S_{ii}} \text{ is defined} \end{aligned}$$

Appreciating the BCE loss from a generalization viewpoint.

For binary classification based on scores:

$$a_i = w^T x^i$$

$$\hat{y} \equiv \operatorname{argmax}(u_i, 1 - u_i)$$

predicted label = 1 if $a_i \geq 0$ —
= 0 otherwise.

$$u_i = \sigma(a_i)$$

* All we care about for accurate classification is that $\operatorname{sign}(w^T x^i)$ match with y_i

* For binary classification another popular notation if $y \in \{+1, -1\}$
 $+1 \Rightarrow$ positive class
 $-1 \rightarrow$ negative class

same as class 0 at true/false notation a_i .

Using ± 1 notation we class that prediction is correct as long as

$$y_i \cdot w^T x_i \equiv y_i \cdot a_i \geq 0$$

0/1 error

almost line

error

Logistic loss for example i with 0/1 class labels.

$$- [y_i \log u_i + (1 - y_i) \log (1 - u_i)]$$

For ± 1 class labels.

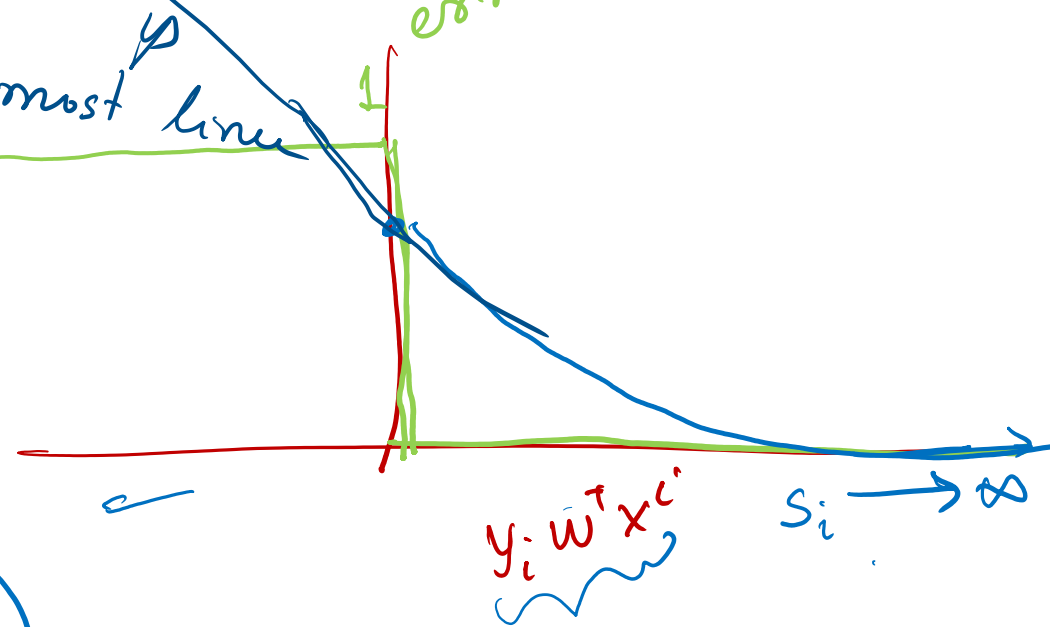
① For -1 class, the loss is

$$-\log(1 - u_i) = -\log\left(1 - \frac{1 - a_i}{1 + e^{-a_i}}\right)$$

$$= -\log\left(\frac{1}{1 + e^{-y_i a_i}}\right)$$

Overall loss:

$$-\log\left(\frac{1}{1 + e^{-a_i y_i}}\right) = \boxed{+ \log(1 + e^{-a_i y_i})} = + \log(1 + e^{-y_i w^T x_i})$$



$$l_i = \log(1 + e^{-s_i}) \quad \text{where} \quad s_i = \mathbf{w}_i^T \mathbf{x}^i$$

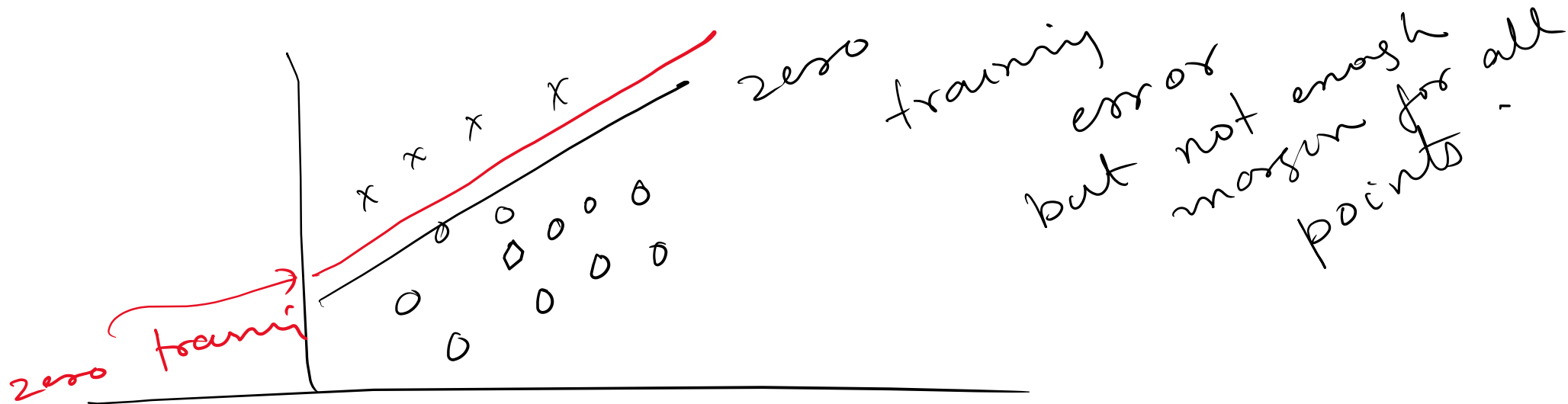
$$s_i \rightarrow \infty, \text{ then } l_i \rightarrow 0$$

$$s_i \rightarrow -\infty \text{ then } l_i \approx \log(1 + e^{-s_i}) \approx -s_i$$

this term dominates

$$s_i = 0 \quad \text{then } l_i = \log 2$$

when a loss function imposes a non-zero penalty even after the score is positively aligned with the class label, we get models which generalize better since they impose a margin



error but with margin.

will generalize better.
- logistic loss has in-built margin.